



Getting Started with Microsoft Foundry

Navigating Microsoft Foundry

Fundamentals of Microsoft Foundry

Hello, friends and welcome to Getting Started with Microsoft Foundry. If you've been watching Microsoft roll out new AI services, it can feel like there's a new name or feature almost every week. Microsoft Foundry, formerly called Azure AI Foundry, is where all of that comes together into one place, so you can actually build, ship, and run AI apps and agents instead of just experimenting with random endpoints. I want you to think of Microsoft Foundry as your AI app and agent factory. It's a unified portal where you design prompts, choose models, connect your own data, orchestrate agents, and keep an eye on usage and safety all in one end-to-end workflow from start to finish. At a high level, you can break Foundry down into four big concepts that you're going to see again and again, models, agents, deployments, and integrations. When you understand these four ideas, the rest of the experience stops feeling a little random and then turns into a workspace for building AI solutions. Let's start with models. In Microsoft Foundry, the model catalog is your menu of choices. It brings together open AI models, Microsoft models, and an expanding list of partner and open models, and it covers things like chat, reasoning, vision, and speech. You can browse, filter, and compare them so that you can pick the one that fits your use case, your performance needs, and your cost and region constraints. Once you've picked a model, you rarely call it directly in a raw one-off way. Instead, you usually wrap it as an agent or an application. Agents in Foundry are designed to orchestrate tools, call other services, work with your data, and handle multi-step workflows, things like analyze this document, then call an internal API with what you found and summarize the results for a human. Foundry gives you an agent service and reusable patterns so you don't have to re-implement that logic every time. To make models and agents usable by real applications, you create deployments. A deployment turns a model configuration into a live callable endpoint with capacity, scaling, and monitoring. You choose the model version, region, and basic limits. Foundry hosts it so your application can send requests reliably and securely. Different deployment options exist for different throughput and control requirements, but they all exist to make your AI workloads production ready. Then there are integrations, how Microsoft Foundry connects with the rest of Azure. You can plug in your apps into data services like Azure Blob Storage, Azure Data Lake, and Azure Cosmos DB, then index that data with Azure AI Search to give your models rich, domain-specific context. You can trigger AI workflows from Azure functions, logic apps, or event-driven services like Event Grid and Service Bus, so AI becomes a part of your existing event pipeline instead of a separate island. On the front end, you can expose your API capabilities through web



apps, APIs, and even Power Platform and Copilot experiences that your users already know. Now you might also notice in the portal and in Microsoft's documentation that there are mentions of a classic Foundry portal. The classic portal was built around Hub and Foundry projects and separate Azure OpenAI resources. The new Microsoft Foundry portal and resource type bring those capabilities together into a single environment where projects live directly under a Foundry resource. It has broader model catalogs, agents, and evaluation tools. And this is the place Microsoft is investing in for new features going forward, and it's what this course will focus on.

Projects and RBAC Basics

To make sense of Microsoft Foundry, you and I need a clean framework of projects and permissions. When you work with Microsoft Foundry, the key idea is where your work lives. Projects are under a Foundry resource. You can think of the Foundry resource as the AI account, and each project is a secure workspace inside that account where you build and run AI apps and agents. A Foundry project is your main unit of organization. It's a container for access management, data upload and integration, and monitoring, all hanging off a single Microsoft Foundry resource. That lets you separate work by use case, customer support, internal knowledge search, or a specific business application without spinning up a new collection of Azure services every time. Inside a project, you'll find the assets you'll use every day, agents, evaluations, datasets, indexes, project files, and the model deployments your applications call. Agents inside of a project share storage and search infrastructure, and you can still bring your own Azure resources when you need stricter compliance or data controls. Because projects are isolated from each other, you can keep development, test, and production separate while still under the same Foundry resource. Each project has its own members, connection to data sources, and observability, so you can tune access per scenario instead of pushing everyone into one giant shared workspace. Permissions for all of this are enforced by Azure role-based access control. And the areas you care about are the account, and that's the Microsoft Foundry resource itself and the project. The built-in AI roles line up neatly with how a real team works. Azure AI Account Owner manages the boundary accounts themselves and can create new ones. Azure AI Project Manager can create and manage projects and has full data actions inside of those projects. Azure AI User is the everyday builder role with reader access to accounts and projects, plus data plane permissions to actually use models, agents, and tools. You can layer standard Owner, Contributor, and Reader roles on top of those when you need broader Azure privileges, but these three AI-specific roles are the core of a least privileged setup. You can map these roles to three people. Your platform or IT admin sits at the subscription or resource group level with Owner or Contributor and gets an Azure AI Account Owner on the Foundry resource so they can create new accounts and control global



settings. Your team lead or project owner gets Azure AI Project Manager on the Foundry resource so they can create projects and manage who's building in them. Your developers and data scientists are assigned Azure AI User on the projects they need, giving them the ability to open the portal, use playgrounds, build agents, and work with evaluations without touching any governance or networking. The last piece of the puzzle is everything outside of Microsoft Foundry. When your projects connect to Blob Storage, Azure AI Search, or Application Insights, the Foundry resources managed identity or a project-specific identity also needs the right roles on those services, such as Storage Blob Data Reader or a suitable search role. If you forget that, the portal looks fine, but data access will fail. Your overall framework becomes aligning role-based access controls or API keys at the Foundry account and project scopes, and then ensuring the managed identities behind your project have matching permissions on any connected Azure resources.

Cost Control, Quotas, and Regions

Cost is often a quiet constraint on AI projects, so I want to give you a simple framework for keeping Microsoft Foundry fast, usable, and affordable. There are three building blocks here, how you build, how quota and rate limit works, and how regions and token budgets fit into that picture. First, billing. Microsoft Foundry is a front door over services like Azure OpenAI, Speech, Vision, and others. You're charged mainly for the usage of those underlying services, often in tokens or units processed, plus supporting resources like Search or Storage. Microsoft's guidance is to estimate spend with the Azure pricing calculator before you build, then track real costs with cost management, budgets, and alerts once your workloads are live. Next up, quotas and rate limits. For models in Foundry, quota is assigned to your subscription on a per region, per model basis, and this is measured as tokens per minute, or TPM. When you create a deployment, you allocate some of that TPM to that deployment, and your remaining quota for that model in that region goes down. So think of TPM as your throughput ceiling. If your app sends more tokens per minute than you've allocated, you'll hit rate limit errors instead of scaling. Now if you outgrow the defaults, you can request higher quota, and for steady, high volume workloads, you can use provision throughput, so capacity is reserved for you instead of just being purely shared. Regions are a part of the story too. Models have specific regional availabilities, and quotas are per region, not global. That means you might deploy the same model in more than one region to spread traffic or to stay close to different users' populations while staying inside of regional limits. It also means quota doesn't move automatically if you switch regions. You need to check which models and capacities exist or you plan to deploy, and you'll want to check that you have confirmed capacity in a given region. Now let's talk about cost guardrails and token budgeting. Every chat interaction is a prompt plus a response, and both are measured in tokens, and that directly drives cost. You



control this by keeping system prompts and examples lean, choosing smaller or cheaper models when they're good enough to work, and avoiding huge documents sent inline when you could use retrieval or chunking. Azure's monitoring guidance emphasizes tracking token usage and latency per deployment using Azure Monitor and Microsoft Boundary observability, so you can see which apps are burning the most tokens and tune them before the costs end up spiking.

Demo: Microsoft Foundry Tour

I want to walk you through three places you'll spend most of your time in Microsoft Foundry, the model catalog and their playgrounds, agents, and tools. To start with, you can see the home page here of the new Microsoft Foundry portal. There isn't much here right now besides the project endpoint and key, and some news about new models. Something important to note, however, is that the new Foundry toggle is turned on to get to this new portal; otherwise, you'll be in what Microsoft calls the classic portal. Where things start to get interesting is when I head up to the Discover tab at the top here and select it. Now you'll see the left-hand navigation pane open, and you'll see Models, Tools, and Solution templates. From here, I'll open up the Model catalog. This is the hub for all of the models you can use in Foundry, Azure OpenAI models, Microsoft models, partner and community models across text, images, audio, and more. You can filter by collection, capability, and more. When I select the model here, I get a details page with information about the model, benchmarks, quick facts, and so much more. On the model page, there's a Deploy button here which I'll click now. You'll see you can choose default settings or pick your own if you want to set the SKU, provision tokens, and more. I'll select Default here, and just like that, it deploys and takes me straight to the playground. On the right here, you'll see the chat conversation pane. On the left, you'll see the configuration, instructions, tools, knowledge, and memory. In the playground, I can send a prompt, see the response, and iterate on the instructions and settings without writing any code. Let's say I have this set up how I'd like. The next step is to save this as an agent, which you can do by clicking up here and then giving it a name, which I'll do now and then hit Create. Now, you may have noticed this interface is a bit different, and that's because Agents is where you design and iterate on full AI agents. You give them instructions, attach tools and data sources, test conversations end to end, and manage versions before publishing them into apps or channels. Now the next really great feature in here is the tools, so I'll go ahead and expand that now. As you can see right away, you can add or upload files. I'll click on Add, and this will bring up the Configured tools page that you can use with your agents, so things like File search, Code interpreter, Azure AI search, Bing Search, Web search, Computer Use, and more just on this page alone. And this is what makes the agent much more powerful. If I select Catalog, it will give you even more tools to choose from and things like



Cosmos DB, Azure Database for Postgres, connection to Atlassian for Confluence and Jira, and if that wasn't enough, you can select Custom where you'll have the option to add your own APIs, MCP servers, and Agent2agent connections.

Setting up a Project and Service Connections for Microsoft Foundry

Demo: Create a Foundry Resource

Hey, friends, and welcome to Project Setup and Service Connections. Now that you've seen the basics, I'll switch to a demo and create the Foundry resource and project. I have the Azure portal pulled up here, and I'm signed into my own personal subscription. You'll want to use a personal subscription if you're following along because the sandbox environment does not support creating these Foundry resources. To get started, I'll search for Foundry here in the search bar and select Microsoft Foundry from the list. Then on the Overview page, I'll click here to create a Foundry resource. On the Basics page, I'll pick my resource group, and I'll call this resource cbps-foundry, and then I'll pick a region. An important note on region, some features are only available on specific regions, so for maximum availability, I'm going to pick East US 2, as that is one of the regions that supports most of the features of Foundry. Now it will prompt me to enter a project name, and I'm just going to call this cb-project. As you might expect, this will create a project within the Foundry resource using that project name. This is looking good, so I'll hit Next. On the Storage page, you can add and configure things like Key Vault, Application Insights, and storage accounts for the speech and language service. These are useful if you know you'll be needing those resources, but you can always add them later if needed. I'll leave these settings as they are and hit Next. In the Networking tab, you can set up inbound access restrictions. All networks works good, so hit Next. On the Identity page, you can choose System assigned or User assigned identities. I'm going to leave this on System assigned, and you'll probably see this information here tell you that this will grant your user the Azure AI User role. That's just fine, so hit Next. With encryption, you can choose to bring your own key, which I don't need to worry about, so hit Next yet again, and on the Tags page, hit Next. Finally, on the Review + submit page, I'll hit Create to get this Foundry resource created. This does take a minute or two to deploy, so I'll be right back. It just finished deploying, and it took about a minute. I'm going to click on the resource group I deployed it into here, and now you can see that it deployed the Foundry resource, as well as the project, which does show up as a separate resource in the Azure portal. If I click on the Foundry resource itself now after it loads, you can see the left-hand navigation menu where you have things like projects, keys, and endpoints, encryption, and more. On the main page is the big go-to Foundry portal, which I'll



select now, and after a few seconds of loading, I'm in the Foundry portal. It has loaded me into the classic portal, and you can tell because the toggle up here is turned off, so I will go ahead and turn that on now, and then it will take me to the new Foundry portal. Right here in the center of the screen, you can see the project endpoint and API key. So when you get to implementing this into an application or code, the details are right here. At the top left, you can see my project name, and if I select it, it will give you a list of recent projects. And if I hit View all resources, it will open another tab that will show you projects and the parent resources for those projects. Now, that's not too useful with just one project and one resource, but as you scale up and out, it is helpful to have this functionality to see all your projects and resources in one spot.

Demo: Connect Azure Services

The real power of Microsoft Foundry shows up when your project and agents start talking to the rest of Azure. I have the new Foundry portal pulled up here along with the Azure portal, and I'm signed into my own personal subscription. Like before, if you're following along, you'll want to use a personal subscription as well. To start with, I'll take a look at the connections I already have by heading to Operate on the top of the window here and then into Admin on the left-hand side. Then I'll click my project and finally head into Connected resources. Unsurprisingly, there's nothing in here at the moment since I haven't connected anything yet, but to add a service to this Foundry project, I'll click on Add connection at the top. Now you can see all of the options you have to connect Azure resources like Azure AI Search, Azure Open API, custom API keys, storage accounts, and more. I'll go ahead and select Azure OpenAI here and hit Continue. You'll see the option to either select a resource or add it manually. I haven't created an OpenAI resource yet, so nothing is showing up. Let me fix that by heading to my Azure portal tab, and in the search bar, I'll search for OpenAI and pick it from the list. Now click Create and select Azure OpenAI. Once it loads, I'll pick my resource group and set my region to the same region as my Foundry resource. Then I'll give it a name and select the Standard pricing tier, then hit Next to move on. Networking looks good, so I'll hit Next and then Next again on Tags. On the final page, I'll hit Create. This does take about a minute to deploy, so I'll be right back when it's done. It's all done deploying now, so I'm going to click on the resource group to take me there. Now I'll head back to the Foundry portal, and if you select the drop-down, you'll see it's not showing yet. So make sure to hit the Refresh button here, click the drop-down again to see the OpenAI resource. Then hit Connect to get this resource added to Foundry. After just a few seconds, you'll see the new connection here, and it's using API keys for authentication. And you can see the key here, and then the endpoint. Something important to note here is that connection details are actually stored in a managed Azure Key Vault that doesn't show up in your subscription, so it's not just saved in plain text or anything. You can bring your own key vault, although



there are some limitations. If you're interested in learning more, I'll put a link to Microsoft documentation for you to take a look at in the Resources section of this course. I want to go ahead and connect Application Insights while I'm here since I'll be using that later, so head back to Add connection and then pick Application Insights and hit Create. Like before, I'll head back to the Azure portal and search for Application Insights and select it. After it loads, click on Create. I'll pick my resource group and give it a name and set the region to the same region as my other resources. Then hit Next and Next again, then Create to get it deployed. I'm going to give this a minute to deploy, so I'll be right back. It's all done deploying now, and I'm going to head back into my Foundry tab here and hit the Refresh button, then select the drop-down and pick that resource, then hit Connect. Great, now I've connected both Application Insights and Azure OpenAI to my Foundry project.

Demo: Managing Keys and Access

Let me show you how to manage keys and access to Microsoft Foundry. I have the Azure portal pulled up, and I'm signed into my own personal subscription. Like before, if you're following along, you'll want to use a personal subscription as well. To get started, I'll open the Foundry resource here and then under Resource Management head into Keys and Endpoints. You can now see the project endpoint and two authorization keys that clients can use when they call this project. This is where you copy values anytime you're wiring up a new app, a test tool, or a script. The reason there are two keys in here is that so you can rotate them without an outage. At any given time, one key is active in your application, and the other is a spare that you can cut over to when you rotate. The guidance is to treat these like any other secret and rotate them regularly, not just when something goes wrong. So let's say my applications are currently using KEY 1. First, I copy KEY 2 and update my app configuration, so requests start authenticating with KEY 2 instead of KEY 1. Once I've confirmed everything is working with KEY 2, I come back here and click Regenerate Key 1 and hit Yes to the prompt. The portal issues a brand-new value, and the old key one stops working. From this point on, KEY 1 is clean and is ready to be the standby key. The next time I rotate, I do the same process in reverse, move clients back to KEY 1, then regenerate KEY 2. As long as you only regenerate the key that your applications are not using, you can rotate as often as you need without breaking running applications. Now let's look at where Microsoft Entra ID and managed identities fit in, because over time, you usually want to rely less on raw keys and more on managed identities. Even though this page shows API keys, the same project endpoints can also accept Microsoft Entra ID tokens. Instead of copying these keys into every app, you give your application or developer identity a role on the Foundry resource, say something like an Azure AI user, and then let the Azure identity libraries fetch tokens for you. Let me show you the basics of how this is configured. I'll head up to the Search bar here and search for



Managed Identities and then select it from the list. Now I'll click on Create to create a user-assigned managed identity. I'm going to pick my resource group here. I'll give this a name of operationsid and set the region to the same region as my other resources, then hit Review + create and Create to make this managed identity. Once it's done creating, I'll go back to my resource group by clicking here and then head into my Foundry project. Once it's open, I'll navigate to Access control, then click on Add and Add role assignment. For the role, I'll search for Azure AI User and then select it from the list and then hit Next. On the Members page, I'll select Managed identity and then Select members. On the right, I'll click the drop-down for Managed identity and pick User-assigned managed identity, which will then show the operationsid managed identity I just created. I'll click it now, and then hit Select. And with that, I'll click Review + assign and click it again to assign the permissions to that managed identity. Now you'll be able to access the Foundry project with that managed identity in your applications.

Building a Generative AI App

Demo: Model Selection and System Prompt Basics

[Autogenerated] Hey friends and welcome to Build, deploy, and operate a generative AI app. Today, you and I are going to pick a good chat model for HR questions, then deploy it and shape a clear system prompt and parameter so it behaves like a careful HR assistant and not a creative storyteller. I have the Azure portal pulled up here, and I'm signed into my own personal subscription. You'll also want to use a personal subscription if you're following along. To get started, I'll navigate up here to Discover and then head into Models to see what options I have for models. As you can see, there are quite a list of options because I want to build an HR assistant that will soon read HR policy documents. I want a general purpose chat model that's strong at reasoning and following instructions, but it doesn't need to be creative. So for that, I'll pick GPT 4.1 here, as this model doesn't require registration like some of the other models, and it's also generally available, so it will work great for my use case. At the top of the page, I'll select deploy and then select Custom Settings to take a look at these settings. First up, you can change the deployment name and type. If I expand the Model version Settings and select the drop-down, you'll see that you can pick specific versions of this model if you need to. You can also change the model version upgrade policy here. This will allow you to control when or if the model is upgraded. You can choose to update automatically or even opt out of updating altogether. Finally, below that, you have the tokens per minute rate limit and the guard rails. This is looking good, so I'll hit deploy to get this model deployed and have it take me to the model playground. On the right is the conversation pane where I'll send test questions, and on the left, I have two key areas I'll focus



on, instructions and parameters, and this is where you turn a generic model into an HR focused assistant. I'll start with the system prompt or instructions. This project is going to use HR policy documents, things like global time off and holidays, travel and expense, remote work and mobility, leave types, and an HR FAQ. Later on we'll attach those files as data sources, but for now I'll write the instructions, assuming that context will be available. You're welcome to write your own prompt here, but to make it a little bit easier, I'm going to paste in a prompt in the instructions box now that I will also provide in the resources section of this course so you can copy and paste it yourself. The prompt I've made is you're a helpful, policy accurate HR assistant for Globalmantic employees. You answer questions using the provided HR documents, global time off and holidays, travel and expense policy, remote work and mobility, leave types and procedures, and HR FAQ and edge cases. Quote or summarize these policies in plain language and tell the employee which document you used. If a question isn't covered by the documents, clearly say you don't know and suggest contacting HR rather than guessing or inventing new policy. That gives the model a clear role, defines its source of truth, and explicitly forbids it from making up new rules, exactly what you want when providing employees answers to HR questions. Now I'll move on to parameters. To see parameters for a model, you can click the settings looking icon here, and now you'll see past messages, max responses, temperature, and top P. For retrieval style Q&A over HR documents, we care more about accuracy and consistency than creative variety. The two settings in here that matter the most are temperature and top P. In Microsoft and OpenAI's guidance, temperature controls how creative or random the model is. Low values make outputs more deterministic and conservative. Higher values make them more diverse and surprising. Or an HR assistant, I don't want surprises. I'll drag the temperature slider down to around 0.2, which keeps responses stable and fact focused. Top P, on the other hand, controls nucleus sampling, the diversity of tokens the model is allowed to choose from. Lower values make responses more predictable, and higher values introduce more variation. Now, Azure's documentation recommends not heavily tuning both of these values at the same time because their interactions can be hard to predict. So for this scenario, I'll leave the top p value at its default and only lower the temperature. If you're following along, make sure to stay on this page and not click anywhere else in the foundry portal, as otherwise your settings will be erased. If you do accidentally click somewhere else, you can always copy my system prompt and set the parameter values again.

Demo: Add Data and Test in the Chat Playground

Today, you're going to take a plain chat model and make it useful for HR policy Q&A by grounding it in a few documents directly inside the Microsoft Foundry chat playground. I have the Azure portal loaded here, and I'm picking right up from where I left off, and that's on the Models tab under Build. I've entered in my



instructions and set the parameters. In another tab, I have the GitHub repo for this course open, and in here you'll find a few HR policy documents that you'll be using with your model to provide grounding and context for HR-related questions. You can find a link to this repo in the Resources section of this course. I'll select this folder and then head into the first policy document here about time off and holiday policy. You'll see that this document includes information about PTO for countries like United States, Germany, UK, and Australia. Scrolling down a bit, you'll see things like company holiday calendar, carryover time, sick leave, etc. There's a lot of information in these documents, so it'll be a great way to utilize AI to help find answers. All right, now I'm going to get these files added to the playground for my model. I've already downloaded these files, so I won't do that on screen, but go ahead and pause the video now and get these files downloaded as you'll be using them soon. I'll head back to the Foundry portal tab and right away under Instructions, you'll see Tools where you can upload files. You might have also noticed there's a section here called Knowledge base, which also grounds the model in your data. If you're wondering what the difference is, Knowledge bases use Azure AI search and are set up for a more complex way of retrieving data, and it scales much better with other systems, and it does have quite a few features. The file upload here under Tools puts your documents in a vector store, which is all managed for you in the background, and it's a great way to get started with a small set of files or a specific use case. For my purposes, the file upload will work great, so I'll click that now. You'll be prompted to create a new vector index, and the default name will work fine, so I'll leave it as it is. Now, I'll drag and drop all the HR document files and wait for them to upload. Once they've been uploaded, I'll click Attach down here to get these added. Great, now the files are attached and ready to go, so I'm going to give this a test. I want to ask a question that should map directly to a policy in the files, so I'll type, How many days off do I get in Germany and do unused days carry over? Then hit Enter to send it. When the response comes back, I'm checking two things. Is the answer concrete and aligned to the policy, and does it show supporting content like a reference or source snippets so I can validate it? Thankfully, it did both. Down at the bottom, you can see references to where it got its data from, and below that is the file reference snippet. So if I highlight over the first one here and hit Read more, it's going to show you exactly what part of the file it got its information from. I'll close out of this and then click here to see all of the references it included, and there are quite a few in here that it was able to pull from. Down at the very bottom of the response, you'll see the model used, how long it took to generate, how many tokens were used, and if I click the small i here, you can see Input and Output response tokens. Last but not least, you can see that I performed a file search as indicated here. So this looks like it's working how I would expect. Now, in order to save my work so far, I need to save this as an agent. I can do this by clicking up here. I'll call this hr-agent and then hit Create. You'll see that the instructions remain along with the files I have attached, and you can also navigate away from the page, and anything you update will now persist under this agent. Let's check



to make sure it works as an agent, and I also want to try and ask it a question that isn't included in the docs to see what it does. So I'll type, What's the performance review process and timeline like, and then hit Enter to send it. After a few seconds, it does respond appropriately by telling me that the provided HR docs do not contain details about the performance review process or any timelines. You may have noticed that the response as an agent has some more info here at the bottom, including AI Quality and Debug. I'll select Debug, and this will pull up the conversation trace and give you information on how exactly the agent got its information. You'll see the input and output, and below you'll see that it used a tool as part of this response, and that was, of course, the file search feature. This conversation trace can be very helpful if you're getting responses you don't expect and you want to track down why it's happening.

Demo: Deploying a Model

Once you have an agent configured, the next step is to deploy it so your applications can interact with it. I have the Foundry portal pulled up here, and I'm picking up right where I left off, and that's on the hr-agent page under Build. In addition, I have the GitHub repo for this course pulled up in another tab and the Azure portal open. A link to the GitHub repo is in the Resources section of this course. Let me show you how to make changes to an agent and how saving in versions work. Let's say you want to make a change to this agent. Maybe you want to remove the reference to the HR FAQ and edge cases document because it no longer applies. Well, you can head up to the Instructions prompt and remove that part of the prompt. Now, unlike the model playground, you'll see the Save button activate in the top right, so I'll click that now. Once it's saved, you might have noticed that left of the Save button, it changed to v2. If I click there, you'll see that every change you make is saved in versions, which allows you to roll back changes to this agent with a single click. You can also compare versions and show version history, which is particularly helpful. As you'd expect if I click version 1 here, you'll see the changes I made to the instructions revert back to include the HR, FAQ, and edge cases document. After you have the agent just how you'd like, now comes the time to preview and deploy it. First up, you can click the Preview button here to see the option to preview the agent and a link to view some sample code. I'll click on Preview agent, and it will open another tab here to the agent preview, which is a nice looking web interface for this agent. If I ask it a question here like What approval is required to work remote from another state, after a few seconds, it will provide the answer and also include the file references here that the end user can click on. So this is great for testing out a final agent. With the agent looking good to go, I'll close out of this tab and get to work on publishing this agent. First up, I'll select the Publish button here and select Publish agent. As the info says, this will make the agent ready for real-world use, and it will have its own URL, identity, access control, etc. That sounds good, so hit Publish here to get this agent published. After about 30 seconds, it will



return and tell you that it has successfully published, and it will give you the Activity Protocol endpoint and Responses API endpoint. Now, just opening the endpoint in the browser won't give you anything useful since this endpoint is designed to be used in code. Now I've prepared a short Python script that will test your endpoint, so I'll work on that now. I'll close out of the Agent published page and then head into my Azure portal tab. Now I'll head into Cloud Shell by selecting the icon up here. I'll expand the window a bit and then select Bash for my environment type. On the Getting started page, I'll select my subscription from the drop-down and hit Apply. After it loads, I'll type touch and then agent.py to create a Python file that I'll drop some code into to test my endpoint. While I'm here, I'll also go ahead and install a requirement my script needs to run, which I'll do by typing pip install openai -user, which will then download and install the OpenAI Python package for me. Next, I need to install the Azure identity packages to allow the script to authenticate to the endpoint, so I'll type pip install azure-identity, and then -user and hit Enter to get that one installed as well. Once it's done, I'll click on Editor here and hit Confirm to switch to the classic Cloud Shell. After it loads, I'll select the bracket icon here, and this will open the editor. I'll select the agent.py file on the left-hand navigation menu to open it in the editor. Now I'm going to head over to the GitHub repo and head to this folder and open the Python script file. Now click on Copy here to copy this code. Then head back into the Azure portal and paste this into the editor. At this point, I need to modify the base URL here to match my endpoint. You'll see some angled brackets here that I've put in so you know where to make the changes. First up is the Foundry name, so I'll type in my Foundry resource name. Next is the project, so I'll enter in my project name, and finally is the agent name, which I'll enter in now. I'll save my changes with Ctrl+S or click over here on the invisible three dots and hit Save. I'm almost ready to give this a test, but I still need to do one more thing, and that's to type az login and then hit Enter to get the access token I need for this script. I'm going to copy the code here first and then click here to log in. On the sign-in page, I'll paste the code and hit Next, then click my account and hit Continue. Now it's logged in and I can close out of this window. You'll see now that the Cloud Shell has updated, and I'll pick my only subscription by hitting Enter. I'm finally ready to give this a run, so I'll type python and agent.py and hit Enter to run it. After just a few seconds, it returns an answer to the question of what public holidays does a German citizen get.

Demo: Monitoring and Alerting

Once your agent has been published, the work doesn't stop there. You still need to keep an eye on it with monitoring and alerting. I'm back in the Azure portal on the HR agent I created earlier, and I also have the Azure portal open in another tab. To get started, I'll open the Monitor tab up here. This is the agent monitoring dashboard, and it's designed to give you a quick, real-time visibility into performance and operational health, specifically token usage and runs, estimated



cost, and error rates. In addition, it will show you any evaluations you have set up for your agent. If you ever land here and see empty charts, the first thing to verify is that this project has an Application Insights resource connected and that your account has at least Reader on that Application Insights resource so the portal can display the monitoring data. Right at the top, you can see Operational metrics that show estimated costs and total token usage. Moving down, you can see how many times the agent was run, and if I scroll a bit, you can see the error rate of this agent. So this is great for a quick dashboard view, but to see more information, I'll head to my Azure portal tab up here and head into the Foundry resource, then expand monitoring and head into Metrics. I'll click on Select metric to see the list of metrics you can look at. First up is HTTP requests, followed by latency information, usage metrics, and so much more. I'll select Processed Prompt Tokens here to see metrics on how many input tokens have been processed, and when I do, you'll see the graph below update. Latency metrics in here can be really useful, so I'll click up here again and scroll up and find the Time Between Tokens, and I'll select it, which will show you the model generation rate in milliseconds. Now, let's say you want to ensure the agent stays performant, and you want to get alerted if the latency goes above 100 ms. What you can do here is click New alert rule, and this will take you to the Create an alert rule window. In the Alert logic, you can set what will trigger this alarm. So if I enter 100 here for the threshold, this is saying that when the average latency is greater than 100 ms, this alert will fire off. If I scroll down, you'll see the When to evaluate section, and this is saying that this rule will run every 5 minutes and look back at the last 5 minutes of data for the average latency. With that looking good, I'll click Next into Actions. Actions is where the rule can send emails, notifications, trigger workflows, or Azure Functions, so it's very powerful. I'll just hit Select action groups to see the default action group that will email any users assigned to the Monitoring Contributor and Monitoring Reader roles. I'll select it and hit Select, then hit Next to Details. On the Details page, you configure the alert rule severity and give it a name, which I'll do now and call it latency-alert. With that complete, I'll hit Review + create, and then hit Create to create this alert. After a few seconds, it will create the alert, and if I head into my resource group and refresh a few times, you'll see the alert rule here set up and ready to go.